



MySQL 8 New Features

The Good, The Bad And The Ugly



Vinicius Grippa

Lead Senior Support Engineer at Percona

Vinicius Grippa is a Percona Senior Support Engineer, Oracle Ace, and author of the book Learning MySQL. Vinicius has a Bachelor's degree in Computer Science and has been working with databases for 16 years. He has experience in designing databases for mission-critical applications and, in the last few years, has become a specialist in MySQL and MongoDB ecosystems. Working in the Support team, he has helped Percona customers with hundreds of different cases featuring a vast range of scenarios and complexities. Vinicius is also active in the OS community, participating in virtual rooms like Slack, and speaking at MeetUps, and presenting conferences in Europe, Asia, North and South America.



Agenda

Agenda

- ❑ DDL improvements
- ❑ Parallel read threads
- ❑ REDO log capabilities (archiving, disabling)
- ❑ EXPLAIN tree format
- ❑ Options and Variables
- ❑ Performance Schema instrumentation
- ❑ DROP / TRUNCATE operations



MySQL 8

MySQL 8

- MySQL 8 Milestone(8.0.0) release 2016-09-12
- MySQL 8 first GA release(8.0.11) is from 2018-04-19
- Why skip versions 6 and 7 and go [straight to 8](#)?
- MySQL is the 2nd in the DB-Engines ranking (behind Oracle)
- Current MySQL GA version 8.0.30 and 8.0.31 being cooked
- Latest Percona Server for MySQL 8.0.29

DDL improvements

ALGORITHM=INSTANT

- Since MySQL 8.0.12, InnoDB supports `ALGORITHM=INSTANT`
- Operations that support `ALGORITHM=INSTANT` only modify metadata in the data dictionary.
- No exclusive metadata locks are taken on the table during preparation and execution phases of the operation, and table data is unaffected, making the operations instantaneous.

ALGORITHM=INSTANT

```
mysql> alter table sbtest2 add g varchar(100) not null default 0,  
algorithm=INSTANT;  
Query OK, 0 rows affected (0.050784 sec)
```

ALGORITHM=INSTANT

Still requires a metadata lock

1007		
97		NULL
585	Waiting for table metadata lock	alter table sbtest1 add column test VARCHAR(200) default '', ALGORITHM=INSTANT, LOCK=DEFAULT
0	init	show processlist

Parallel Threads for Online DDL Operations

- The number of parallel threads that can be used to scan clustered index is defined by the `innodb_parallel_read_threads` variable. The default setting is 4.
- The number of parallel threads that sort and load data is controlled by the `innodb_ddl_threads` variable, introduced in MySQL 8.0.27. The default setting is 4.
- The new `innodb_ddl_buffer_size` variable defines the maximum buffer size for DDL operations.

Defining a buffer size limit avoids potential out of memory errors for online DDL operations that create or rebuild secondary indexes.

Parallel Threads for Online DDL Operations

```
mysql> alter table sbtest2 add g varchar(100) not null default 0,  
algorithm=inplace;  
Query OK, 0 rows affected (1 min 13.90 sec)
```

```
mysql> set session innodb_ddl_threads=16;  
mysql> set session innodb_parallel_read_threads=16;  
mysql> alter table sbtest2 add g varchar(100) not null default 0,  
algorithm=inplace;  
Query OK, 0 rows affected (41.66 sec)
```

Parallel Read Threads

Parallel Read Threads

- MySQL 8.0.14 has (for now limited) an ability to perform parallel query execution. The `innodb_parallel_read_threads`(default 4) defines the number of threads for parallel index reads.
- It is limited to select count(*) from table queries as well as check table queries.

Parallel Read Threads

```
mysql> set session innodb_parallel_read_threads=1;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select count(*) from joint;
```

```
+-----+  
| count(*) |  
+-----+  
| 134217728 |  
+-----+
```

```
1 row in set (15.29 sec)
```

Parallel Read Threads

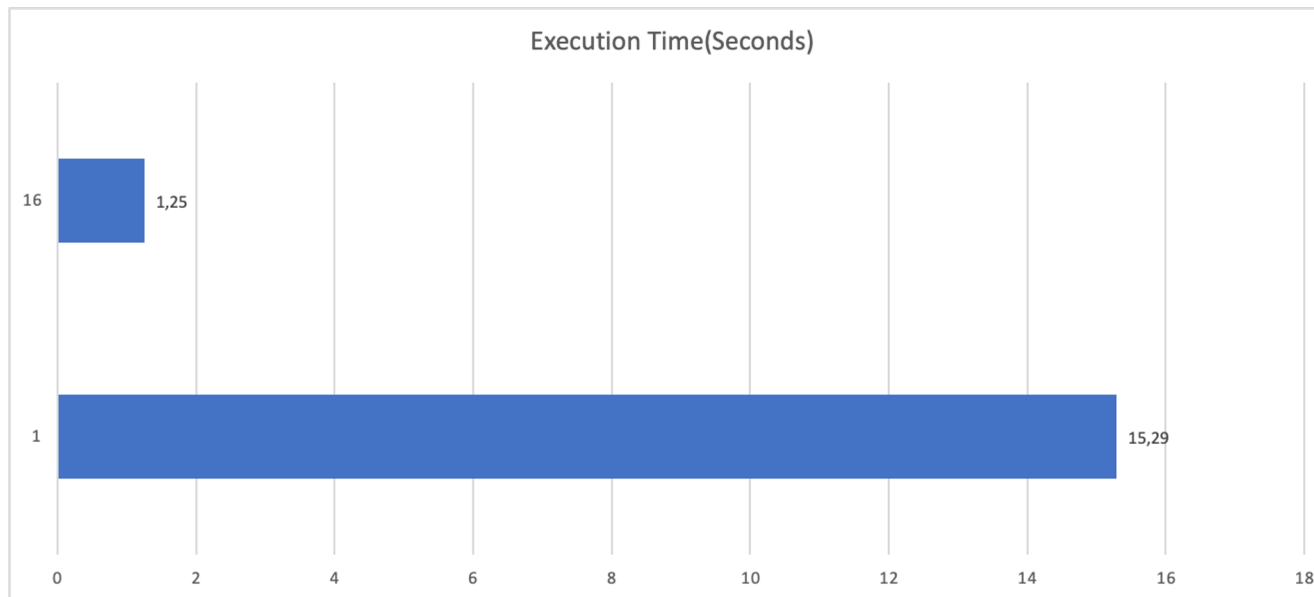
```
mysql> set session innodb_parallel_read_threads=16;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select count(*) from joint;
```

```
+-----+  
| count(*) |  
+-----+  
| 134217728 |  
+-----+
```

```
1 row in set (1.25 sec)
```

Parallel Read Threads



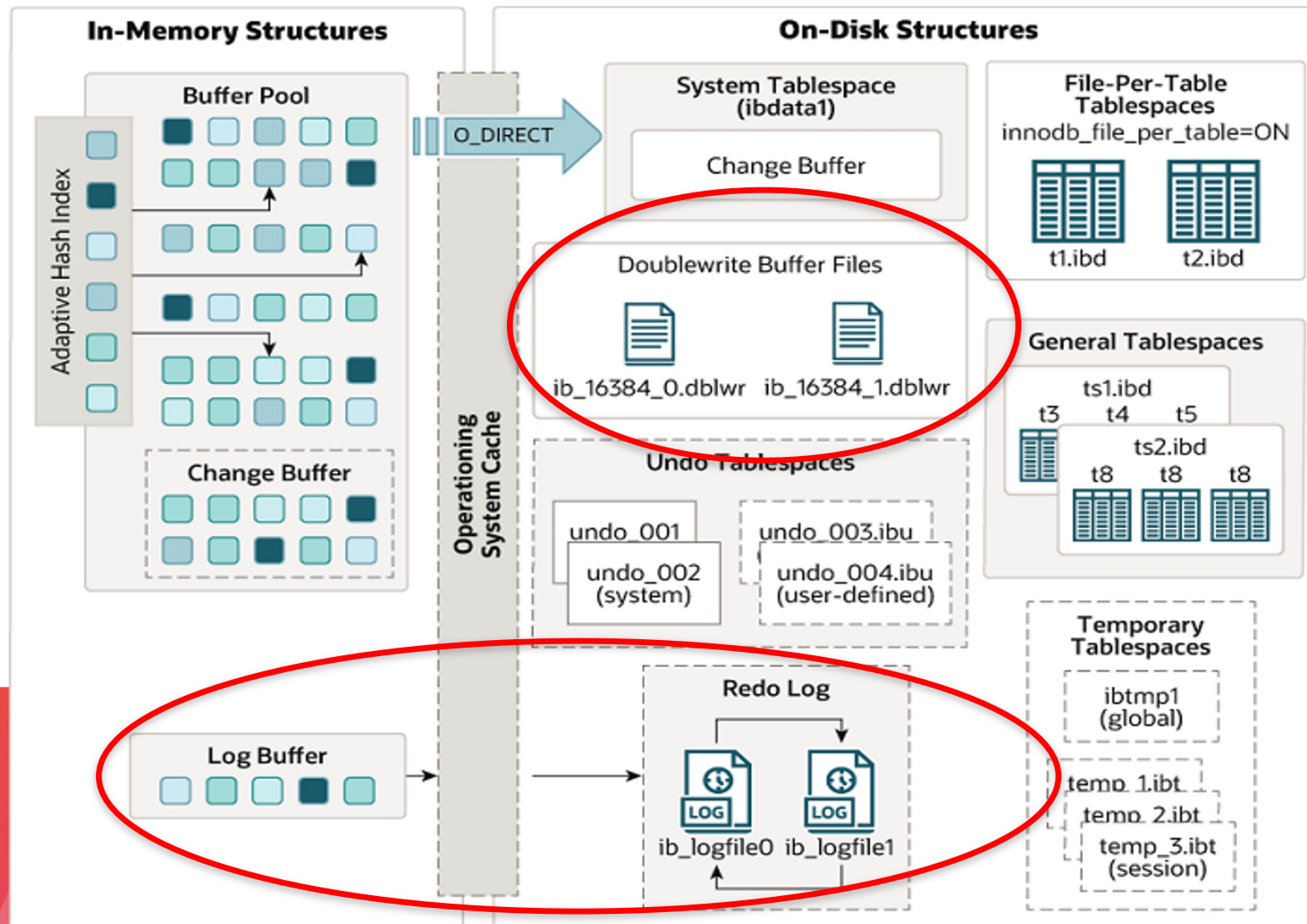
REDO log capabilities

Redo Log Archiving

- MySQL 8.0.17 releases the redo log archiving(`innodb_redo_log_archive_dirs`).
- Backup utilities that copy redo log records may sometimes fail to keep pace with redo log generation while a backup operation is in progress, resulting in lost redo log records due to those records being overwritten.

Disabling Redo Log

- As of MySQL 8.0.21, we can disable redo logging using the `ALTER INSTANCE DISABLE INNODB REDO_LOG` statement.
- This functionality is intended for loading data into a new MySQL instance. Disabling redo logging speeds up data loading by avoiding redo log writes and doublewrite buffering.



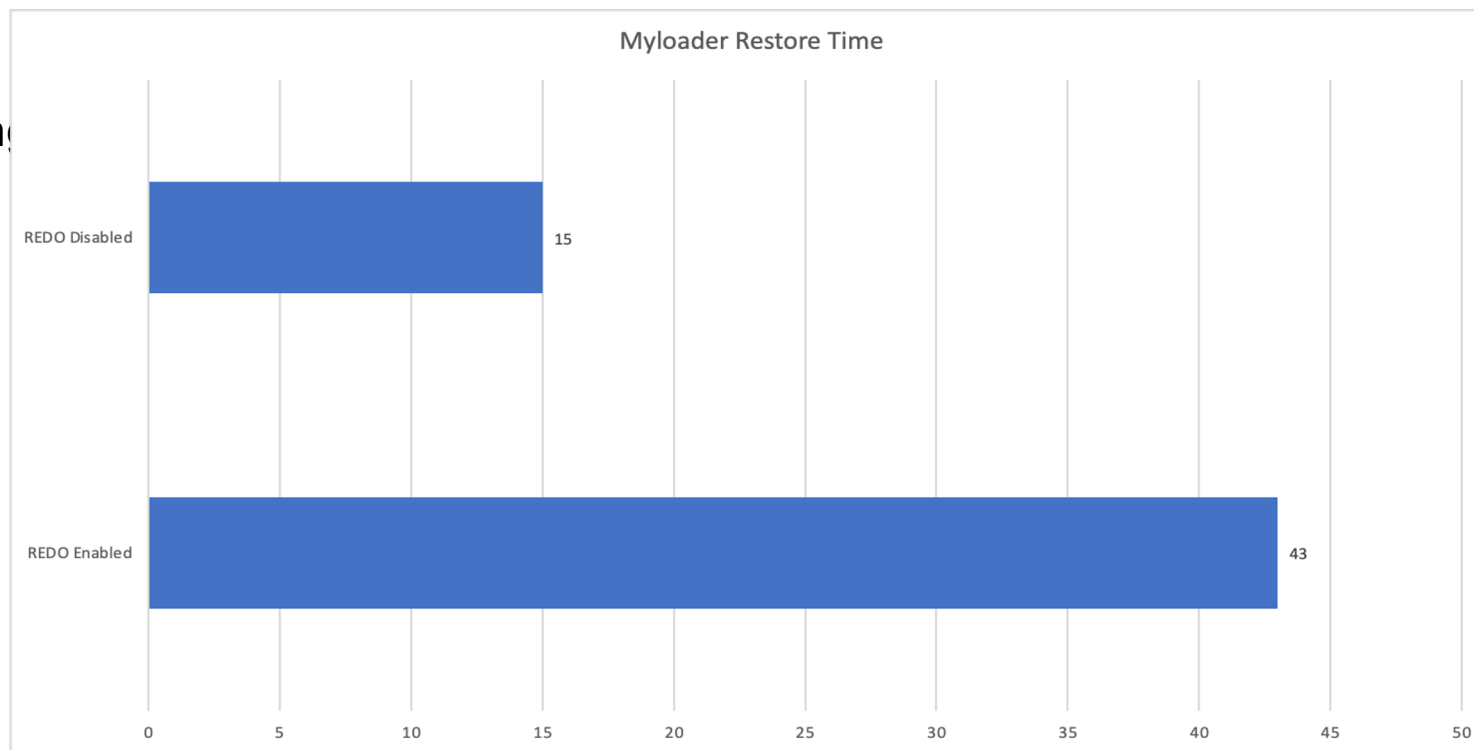
Disabling Redo Log - Benchmark

- **Hardware**
- c5.4xlarge
- 16 CPU
- 32 GB RAM
- 400GB provisioned disk with 3000 IOPs

Disabling Redo Log - Benchmark

- **MySQL 8.0.29**
- 20GB of InnoDB Buffer Pool
- 20GB of redo log size
- sync_binlog=0
- innodb_flush_log_at_trx_commit=0
- ~95GB Data Set

Disabling



Dedicated log writer threads

- As of **MySQL 8.0.22**, you can enable or disable log writer threads using the `innodb_log_writer_threads` variable(default ON).
- Dedicated log writer threads can improve performance on high-concurrency systems, but for low-concurrency systems, disabling dedicated log writer threads provides better performance.

EXPLAIN tree format

Explain Tree Format

```
EXPLAIN FORMAT=TREE
SELECT first_name, last_name, SUM(amount) AS total
FROM staff INNER JOIN payment
  ON staff.staff_id = payment.staff_id
  AND
  payment_date LIKE '2005-08%'
GROUP BY first_name, last_name;
```

- As of MySQL 8.0.21, query execution information is displayed using the TREE output format.

```
-> Table scan on <temporary>
  -> Aggregate using temporary table
    -> Nested loop inner join (cost=1757.30 rows=1787)
      -> Table scan on staff (cost=3.20 rows=2)
        -> Filter: (payment.payment_date like '2005-08%') (cost=117.43 rows=894)
          -> Index lookup on payment using idx_fk_staff_id (staff_id=staff.staff_id)
            (cost=117.43 rows=8043)
```

Explain Tree Format

EXPLAIN **ANALYZE**

```
SELECT first_name, last_name, SUM(amount) AS total
FROM staff INNER JOIN payment
  ON staff.staff_id = payment.staff_id
  AND
  payment_date LIKE '2005-08%'
GROUP BY first_name, last_name;
```

-> Table scan on <temporary> **(actual time=0.001..0.001 rows=2 loops=1)**

-> Aggregate using temporary table (actual time=58.104..58.104 rows=2 loops=1)

-> Nested loop inner join (cost=1757.30 rows=1787) **(actual time=0.816..46.135 rows=5687 loops=1)**

-> Table scan on staff (cost=3.20 rows=2) **(actual time=0.047..0.051 rows=2 loops=1)**

-> Filter: (payment.payment_date like '2005-08%') (cost=117.43 rows=894) (actual time=0.464..22.767 rows=2844 loops=2)

-> Index lookup on payment using idx_fk_staff_id (staff_id=staff.staff_id) (cost=117.43 rows=8043) (actual time=0.450..19.988 rows=8024 loops=2)

Explain Tree Format

There are several new measurements here:

- Actual time to get first row (in milliseconds)
- Actual time to get all rows (in milliseconds)
- Actual number of rows read
- Actual number of loops

Filter: (payment.payment_date like '2005-08%')
(cost=117.43 **rows=894**)
(actual time=0.464..22.767 **rows=2844 loops=2**)

```
1 EXPLAIN ANALYZE
2 SELECT first_name, last_name, SUM(amount) AS total
3 FROM staff INNER JOIN payment
4   ON staff.staff_id = payment.staff_id
5   AND
6   payment_date LIKE '2005-08%'
7 GROUP BY first_name, last_name;
8
9 -> Table scan on <temporary> (actual time=0.001..0.001 rows=2 loops=1)
10    -> Aggregate using temporary table (actual time=58.104..58.104 rows=2 loops=1)
11    -> Nested loop inner join (cost=1757.30 rows=1787) (actual time=0.816..46.135 rows=5687
loops=1)
12    -> Table scan on staff (cost=3.20 rows=2) (actual time=0.047..0.051 rows=2 loops=1)
13    -> Filter: (payment.payment_date like '2005-08%') (cost=117.43 rows=894) (actual
time=0.464..22.767 rows=2844 loops=2)
14    -> Index lookup on payment using idx_fk_staff_id (staff_id=staff.staff_id)
(cost=117.43 rows=8043) (actual time=0.450..19.988 rows=8024 loops=2)
```

Explain Tree Format

What can we do with this information?

- If you wonder what it's taking so long, look at the timing. Where does the executor spend its time?
- If you wonder why the optimizer chose that plan, look at the row counters. A large difference (i.e., a couple of orders of magnitude or more) between the estimated number of rows and the actual number of rows is a sign that you should look closer at it. The optimizer chooses its plan based on the estimate, but looking at the actual execution may tell you that another plan would have been better.

Options and Variables

innodb_dedicated_server

- Introduced in MySQL 8.0.3

When `innodb_dedicated_server` is enabled, InnoDB automatically configures the following variables:

- `innodb_buffer_pool_size`
- `innodb_log_file_size`
- `innodb_log_files_in_group` (as of MySQL 8.0.14)
- `innodb_flush_method`

innodb_buffer_pool_in_core_file

- Introduced in MySQL 8.0.14
- Disabling the `innodb_buffer_pool_in_core_file` variable reduces the size of core files by excluding InnoDB buffer pool pages.

innodb_use_fdatasync

- Introduced in MySQL 8.0.26
- Allows MySQL to use `ftadasybc()` instead of `fsync()`.
- An `fdatasync()` call does not flush changes to file metadata unless required for subsequent data retrieval, providing a potential performance benefit.

innodb_deadlock_detect

- Introduced in MySQL 8.0
- This option is used to disable deadlock detection.

binlog_expire_logs_auto_purge

- Introduced in MySQL 8.0.29
- Enables or disables automatic purging of binary log files (default ON).
- This takes precedence over any setting for `binlog_expire_logs_seconds`.

global_connection_memory_limit

- Introduced in MySQL 8.0.28
- The default is virtually infinite (**16 EB**)
- Set the total amount of memory that can be used by all user connections (including `root` user)

global_connection_memory_tracking

- Introduced in MySQL 8.0.28
- Determines whether the server calculates `Global_connection_memory` (Default `FALSE`).

Performance Schema instrumentation



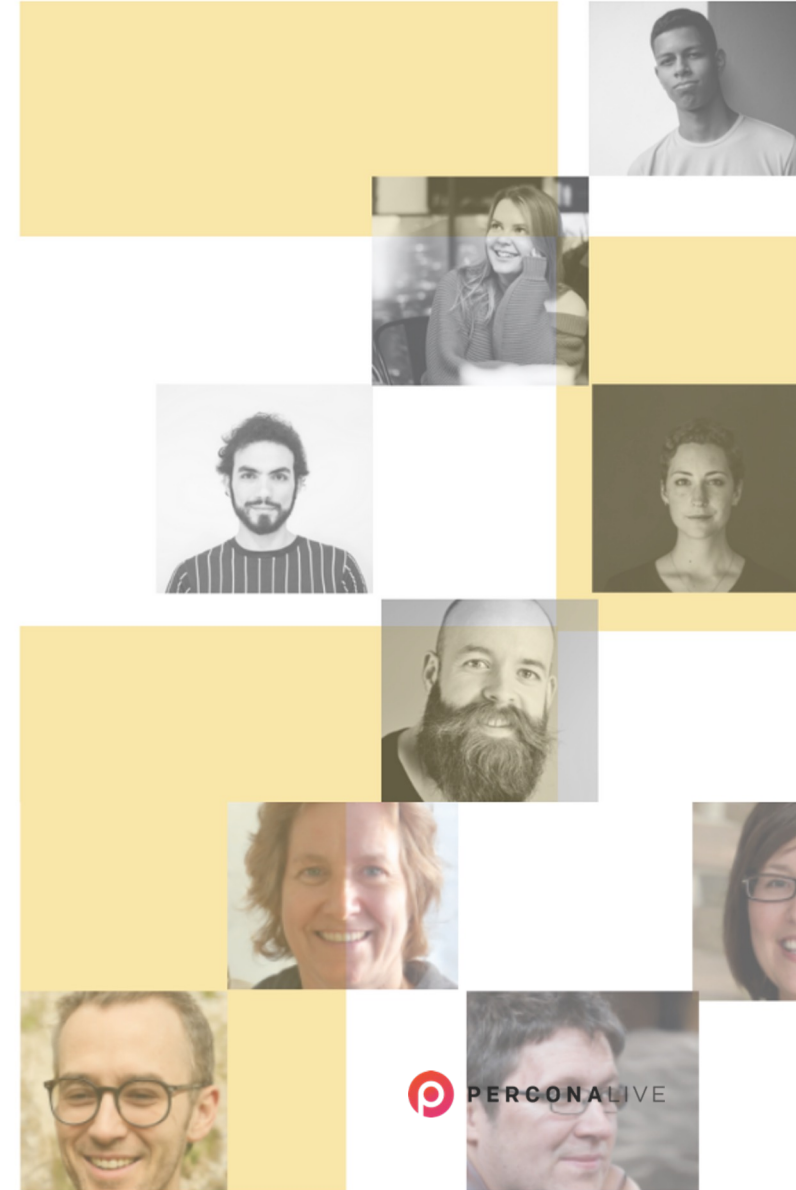
We're hiring
Join our team!

OPEN POSITIONS



#RemoteWork

APPLY NOW: percona.com/careers



谢谢

Thank you

Grazie

Obrigado

Gracias

percona.com/contact | info@percona.com

vinicius.grippa@percona.com

